

Yuelin Liu

Software Engineer — Full-Stack (TypeScript · React · Node.js)

Melbourne, VIC (Open to Relocation / Remote) · 0451 690 105 · liuyuelintop@gmail.com · liuyuelin.dev · github.com/liuyuelintop
· linkedin.com/in/liuyuelintop

Full Australian work rights · Temporary Graduate visa (subclass 485) to March 2029 · no sponsorship required

PROFILE

Full-stack engineer who works from the user's problem down to the architecture, then proves the result with tests. At an AI SaaS startup I traced a 26.7s API path to its slowest dependencies and cut it to 5.6s. Since then I have designed and built five products end to end, including an AI drafting engine that cannot cite experience its user has not confirmed, a privacy-first wage tool for hourly workers, and an open-source plugin published on npm.

TECHNICAL SKILLS

Languages & Core: TypeScript, JavaScript (ESNext), Python, SQL, HTML5/CSS3

Product Engineering: React 19, Next.js (App Router), Node.js, Express, REST & BFF APIs, MongoDB, Convex, IndexedDB/Dexie

Quality & Delivery: Playwright (E2E & Headless), Vitest, Jest, Supertest, Zod Contracts, GitHub Actions CI/CD, Docker, Vercel, Tailscale, NGINX

AI Integration: OpenAI Function Calling; Anthropic, Gemini and DeepSeek APIs; Structured Outputs; Evidence Grounding & Fact-Checking Pipelines

EXPERIENCE

Independent Full-Stack Developer — Melbourne Tech Consultancy (Contract)

Sep 2025 – Present

Problem: the consultancy needed working demonstrations, not mock-ups, to pitch prospective clients whose briefs were informal and still changing.

- Turned informal briefs from prospective specialty-coffee and drone-training clients into working, authenticated, database-backed prototypes that the consultancy demonstrated in its client pitches.
- Owned each build end to end on short deadlines: data model, sign-in (Clerk OAuth), real-time backend (Convex) and Vercel deployment on Next.js 15, reshaping scope and schema as requirements firmed up.

Full-Stack Developer — ByteCroniX (early-stage AI SaaS)

Mar 2025 – Jun 2025

Problem: an inherited points-summary request took almost 27 seconds, with no visibility into where the time went.

- **Cut the request from 26.7s to 5.6s (79% faster) in integration testing:** added timing instrumentation, identified the scoring services as the dominant cost, then ran the independent service calls in parallel instead of in sequence.
- Implemented OpenAI function-calling flows in the Node.js/Express backend for AI-assisted product interactions, delivered through BFF routes shared by the web, Android and iOS clients.
- Worked alongside senior engineers on API testing (Jest, Supertest, Postman), NGINX troubleshooting and multi-service CI/CD.

PRODUCTS & PROJECTS

Marshal — Evidence-grounded job-application engine and pipeline

Sep 2026 – Present

Problem: AI writing tools state experience the applicant never had, and "review it carefully" fails by the tenth application of the week. Meanwhile applications scatter across portals and devices.

- **Check before generation, not after:** Experience is recorded once as facts, each with a claim and a boundary (what it must not imply), confirmed per use. The server removes unconfirmed and out-of-scope facts before prompt assembly, so the guarantee never rests on prompt wording.
- **Deterministic verification:** A sentence-licensing pass flags any generated sentence whose numbers or names are absent from the facts it cites. SHA-256 fingerprints of each step's inputs mark triage, drafts and résumés stale the moment a fact changes.
- **One workflow, end to end:** Capture → requirement-by-requirement triage → fact-checked cover letter → A4 résumé and letter PDFs (offline headless Chromium, CJK-safe) → pipeline board with a due-action queue through to offer.
- **Private by network:** Self-hosted in Docker behind a Tailscale tailnet with no public port. Laptop-to-phone handoff uses revision-checked saves that report a conflict instead of overwriting newer work.
- **Proof:** 220 unit and contract tests, 8 browser journey suites, a Docker PDF-rendering check and a full-history secret scan pass on every push in GitHub Actions. (TypeScript, React 19, Node.js, Playwright, Zod)

Job Search Dispatch — Job-ad ingestion gateway and browser bridge

Jul 2026 – Present

Problem: job boards refuse server-side fetches, and pasting long ads by hand is slow and lossy. Once the ad is in, model calls across providers hang, time out or drift without anyone noticing.

- **Dual-channel ingestion:** A Manifest V3 extension reads the rendered ad from the user's own open tab; a headless Chromium fallback fetches by URL behind SSRF request interception and DNS pinning. On the deployed service a live SEEK ad extracted in 1.3s.
- **Dependable model gateway:** Bring-your-own-key routing across Anthropic, OpenAI, Gemini and DeepSeek, with trace IDs, server-owned deadlines and cancellation that propagates to the upstream provider call.
- **Release discipline:** Runs as a managed macOS LaunchAgent. Every release is gated by 614 deterministic tests and 40 browser flows, and a drift test fails when a prompt reads any field its declared input contract omits. Its evidence model was the prototype that Marshal rebuilds as a multi-device product.

MoneyGuard — Local-first wage and cashflow tool for hourly workers

Jun 2026 – Aug 2026

Problem: hourly and casual workers are paid from timecards and cannot easily see what a week's shifts leave after expenses, or whether their cash will last until payday.

- **Photo to decision:** Timecard photo → reviewed extraction → Zod-validated data → pure, deterministic wage, burn and surplus calculations → payday forecast (safe / caution / critical), buffer planner and a "cost in labour hours" simulator.
- **Privacy as a product decision:** Financial history stays in the browser (IndexedDB). The optional AI explanation receives only masked weekly metrics, never images, line items or identifiers, and vision OCR runs only with explicit consent.
- **Runs without secrets:** Fixture providers let the whole product and its Vitest and Playwright suites run offline with no API keys. (Next.js 16, React 19, TypeScript, Dexie)

DSH Conversation Exporter — Open-source developer plugin on npm

Jul 2026 – Aug 2026

Problem: DeepSeek Harness exports a session only as a raw event log (a ZIP of JSONL), which suits replay but not reading, version control or handing a conversation to another assistant.

- Published an MIT-licensed npm package (v0.3) that adds one-click **Export Chat:** clean Markdown of the human messages and final answers, with optional whole-turn selection, leaving the official log untouched.
- Filters on the host before export, dropping reasoning, tool activity and runtime metadata while preserving Markdown and Unicode. Release checks cover tests, syntax and an npm dry run. Listed in the Awesome DSH Plugin catalogue.

Melbourne University Ultimate Club — Operations platform

Jul 2025 (hardened Aug 2026)

Problem: the club ran rosters, tournament selection and events across spreadsheets and group chats.

- Built a Next.js 15 / TypeScript / MongoDB platform covering public, member and admin workflows: roster and tournament selection, events, announcements and video.
- In a 2026 hardening pass, centralised authorisation on the server around database-backed roles, turned each write-path defect found into a regression test, and enforced them in GitHub Actions CI.

EDUCATION & CERTIFICATIONS

The University of Melbourne | Master of Information Technology

2022 – 2023

Melbourne, Australia

University of Nottingham Ningbo China | BSc (Hons) Computer Science

2016 – 2020

Ningbo, China

Agentic AI Engineering on AWS | Udemy (Ed Donner)

Completed Nov 2025

Built an event-driven multi-agent system on AWS using Bedrock (Claude), Lambda, SQS, and Aurora Serverless v2 provisioned via modular Terraform.